Understand your database.

A step-by-step reviewer for learning how Analista stores users, thesis projects, PDFs, and evaluator reviews.

29	275	35
tables	columns	foreign keys

Learn these words first

Term	What it means
Database	The collection of tables used by the system: `analista`.
Table	A collection of related records, such as `users` or `projects`.
Column	One field, such as `name`, `title`, or `status`.
Row	One saved record: one person, one project, or one review request.
Primary key (PK)	A value that identifies one row. Often `id`.
Foreign key (FK)	A column whose value points to a key in another table.
NULL	A value is absent. It is different from zero or an empty string.
JOIN	An SQL operation that combines records using a connection between them.

Columns go down; records go across

This fictional example represents two rows in `projects`:

id	title	user_id	status
501	Campus Library System	101	approved
502	Campus Parking App	101	draft

Both projects belong to the account with `users.id = 101`. The title is a column; each complete project is a row.

> **Your study order**
>
> Read lessons 1–6, try the SELECT queries in phpMyAdmin, then study the seven diagrams at the end. The diagram pages are larger so the full columns stay readable when you zoom in.

This PDF contains schema information and fictional teaching examples. It does not include live account records, passwords, private messages, or thesis files.

Follow the student-to-review connection.

Accounts, projects, documents, and reviews are separate tables connected through IDs.

Student account → Project → PDF version → Review request

An evaluator account connects to the review request through `evaluator_id`.

Column storing the connection	Points to	Meaning
`projects.user_id`	`users.id`	Project owner
`project_user.project_id`	`projects.id`	Group membership: project
`project_user.user_id`	`users.id`	Group membership: account
`document_versions.project_id`	`projects.id`	Project containing the PDF
`document_versions.user_id`	`users.id`	PDF uploader
`review_requests.project_id`	`projects.id`	Project being reviewed
`review_requests.document_version_id`	`document_versions.id`	Exact PDF record being reviewed
`review_requests.evaluator_id`	`users.id`	Assigned evaluator
`review_requests.requested_by`	`users.id`	Account requesting the review
`feedback.review_request_id`	`review_requests.id`	Review receiving the feedback
`feedback.user_id`	`users.id`	Feedback author

Read this fictional example

- `users`: ID **101** is Ana (student); ID **201** is Ben (evaluator).
- `projects`: ID **501** is “Campus Library System”; `user_id = 101`.
- `document_versions`: ID **701**, `project_id = 501`, `version = 1`.
- `review_requests`: ID **901** connects project **501**, PDF record **701**, and evaluator **201**.

> **Two important differences**
>
> The PDF record ID (`701`) identifies a row; the version number (`1`) identifies its version within a project. Also, a review status records its workflow state; it is not academic approval from an adviser or panel.

Start with SELECT.

Open **phpMyAdmin** → **analista** → **SQL**. Run one query at a time. These examples read data; they do not change or delete records.

1. Display user IDs, names, and roles

```
SELECT id, name, role
FROM users
ORDER BY id;
```

`SELECT` chooses columns; `FROM` chooses the table; `ORDER BY` sorts the result.

2. Display students only

```
SELECT id, name
FROM users
WHERE role = 'student'
ORDER BY name;
```

`WHERE` keeps only rows that match a condition. Text values use single quotes.

3. Display each project with its owner's name

```
SELECT p.id, p.title, u.name AS student_name, p.status
FROM projects AS p
JOIN users AS u ON u.id = p.user_id
ORDER BY p.id;
```

`p` and `u` are short table names called aliases. `AS student_name` gives the output column a clear label.

4. Display uploaded PDF versions

```
SELECT p.title, dv.id AS pdf_record_id,
       dv.version, dv.original_name
FROM document_versions AS dv
JOIN projects AS p ON p.id = dv.project_id
ORDER BY p.id, dv.version;
```

> **Practice**
>
> Change the second query to `WHERE role = 'evaluator'`. Predict which accounts will appear before you run it.

Connect names to review requests.

The review request stores linked IDs. JOIN retrieves the names and title from their original tables.

```
SELECT
    rr.id AS review_id,
    evaluator.name AS evaluator_name,
    p.title AS thesis_title,
    student.name AS student_name,
    dv.version AS pdf_version,
    rr.status
FROM review_requests AS rr
JOIN users AS evaluator ON evaluator.id = rr.evaluator_id
JOIN projects AS p ON p.id = rr.project_id
JOIN users AS student ON student.id = p.user_id
JOIN document_versions AS dv
    ON dv.id = rr.document_version_id
ORDER BY rr.id;
```

Why is users joined twice?

The same `users` table stores all account roles. The alias `evaluator` follows `rr.evaluator_id`; the alias `student` follows the project's owner ID. No separate student or evaluator name column is needed in `review_requests`.

`student_name` in this query is the **project owner's name**. Additional group members are stored through `project_user`.

Expected result for the teaching example

review_id	evaluator_name	thesis_title	student_name	pdf_version	status
901	Ben	Campus Library System	Ana	1	accepted

Review status values

Status	Meaning
pending	Waiting for the evaluator to respond.
accepted	The evaluator accepted and can review the assigned PDF.
declined	The evaluator declined the request.
completed	The evaluator submitted the completed review.

A JOIN displays connected rows. Foreign keys enforce that referenced records exist; application rules also enforce roles, permissions, capacity, and project consistency.

Count records and check your understanding.

5. Count projects for each student, including zero

```
SELECT u.id, u.name, COUNT(p.id) AS project_count
FROM users AS u
LEFT JOIN projects AS p ON p.user_id = u.id
WHERE u.role = 'student'
GROUP BY u.id, u.name
ORDER BY u.name;
```

`LEFT JOIN` keeps students who have no project. `COUNT(p.id)` counts actual project rows, so a student without a project gets zero.

6. Count reviews by status

```
SELECT status, COUNT(*) AS review_count
FROM review_requests
GROUP BY status
ORDER BY status;
```

`COUNT(*)` counts rows. `GROUP BY` combines rows with the same status into a group.

Quick review questions

Question	Answer
Where are account names stored?	`users.name`.
Where is the thesis title stored?	`projects.title`.
Which column identifies the project owner?	`projects.user_id` → `users.id`.
Where are additional project members stored?	`project_user`.
How is a review tied to its exact PDF?	`review_requests.document_version_id` → `document_versions.id`.
Does a revised upload replace the old PDF row?	No. It creates another document version. Earlier reviews keep their original PDF reference.
Does the PDF itself live inside a normal database row?	The file lives in storage. The database stores its path and metadata.
Why can a review query display names without name columns?	JOIN follows stored IDs to the corresponding rows in users.

Know what each table stores.

Table	Purpose
users	Accounts, roles, profiles, and evaluator availability.
projects	Thesis details, owner, visibility, and review settings.
project_user	Additional student group members for projects.
document_versions	Uploaded thesis PDF versions and file metadata.
review_requests	Project, exact PDF, evaluator, status, and final review summary.
feedback	Structured feedback and conversation entries for reviews.
ratings	Ratings and comments linked to completed reviews.
evaluator_verifications	Evaluator applications, profile snapshots, and decisions.
evaluator_evidence	Private supporting files for evaluator applications.
invitations	Expiring evaluator invitation records.
direct_conversations	The two participants in a private conversation.
direct_messages	Private conversation messages and read status.
direct_message_attachments	Files attached to private messages.
community_chat_messages	Shared community chat messages.
community_chat_presences	Community chat presence records.
tutorials	Learning resources and publication details.
tutorial_user	Tutorial bookmarks saved by members.
activities	In-app activity notifications and read status.
admin_actions	Administrator action history and decision context.
reports	Community reports and their resolution.
sensitive_file_accesses	Access history for protected files.
sessions	Laravel sign-in session data.
password_reset_tokens	Password recovery tokens.
cache	Cached application values.
cache_locks	Locks used by the cache system.
jobs	Queued background jobs.
job_batches	Grouped background-job execution information.
failed_jobs	Failed background jobs.
migrations	Database migrations already applied.

Diagram key: PK = primary key · FK → = enforced foreign key · UQ = unique column/index · REF ⇢ = application reference without an enforced FK · * = nullable column.

MariaDB stores some application JSON as LONGTEXT and booleans as TINYINT(1). Next: seven diagrams, from the core flow to the complete column map.

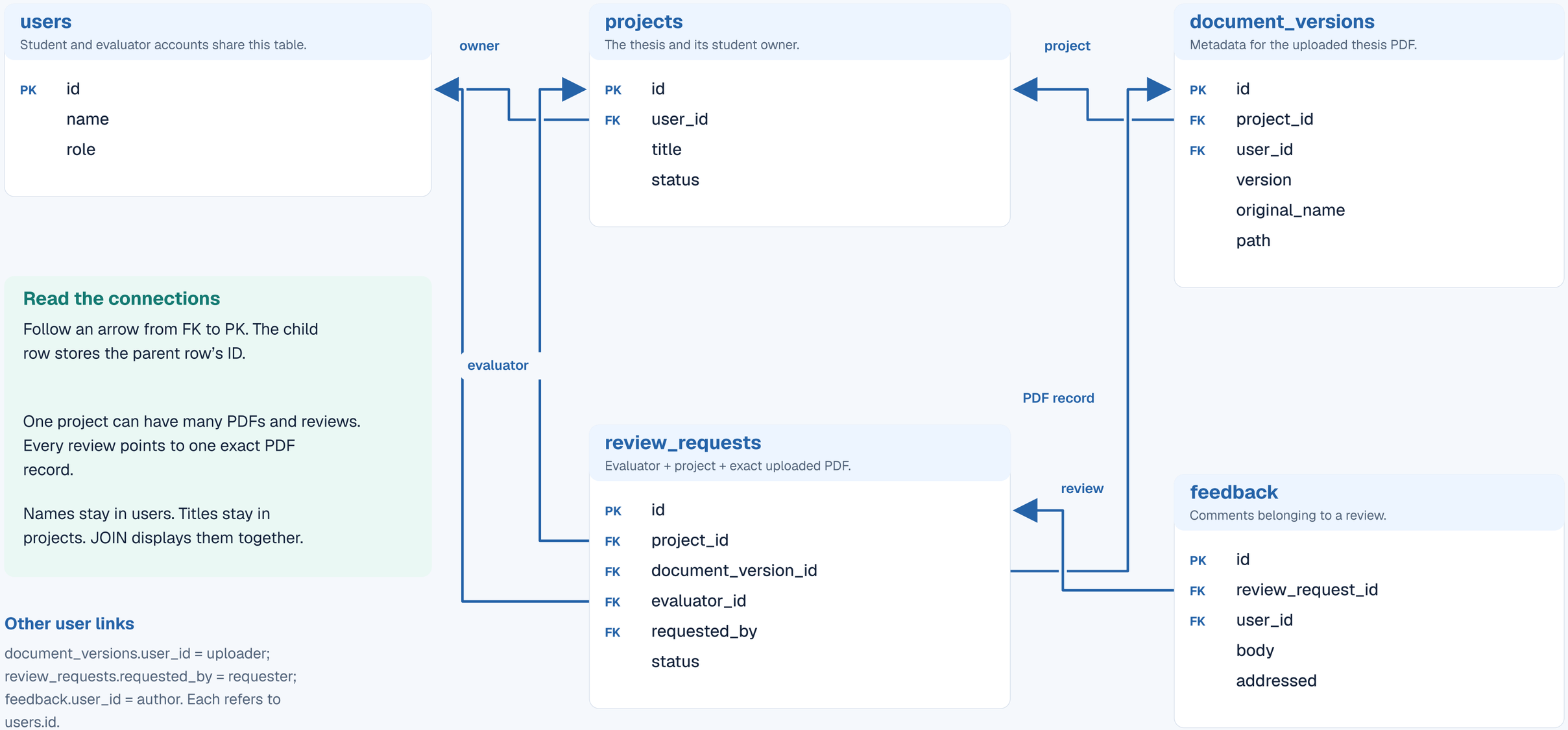
How the main records connect

Key columns shown. Arrow direction: foreign-key column → referenced primary key. Full columns are on the next sheets.

PK Primary key: unique row ID **FK → Enforced foreign key: child column → parent key**

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index



Read the connections

Follow an arrow from FK to PK. The child row stores the parent row's ID.

One project can have many PDFs and reviews. Every review points to one exact PDF record.

Names stay in users. Titles stay in projects. JOIN displays them together.

Other user links

document_versions.user_id = uploader; review_requests.requested_by = requester; feedback.user_id = author. Each refers to users.id.

What are columns and rows?

Fictional teaching example only. IDs below are not copied from your accounts.

id	title	user_id	status
501	Campus Library System	101	approved
502	Campus Parking App	101	pending

In this projects example, each vertical field is a COLUMN and each horizontal record is a ROW. Both projects belong to the student with users.id = 101.

Accounts, projects and uploaded PDFs

Full columns • read users first, then projects, then document_versions.

PK Primary key: unique row ID FK → Enforced foreign key: child column → parent key

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

users

All accounts: students, evaluators and administrators.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	name	VARCHAR(255)	
UQ	email	VARCHAR(255)	
	email_verified_at *	TIMESTAMP	
	password *	VARCHAR(255)	
	remember_token *	VARCHAR(100)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	role	VARCHAR(255)	
	is_suspended	TINYINT(1)	
	headline *	VARCHAR(255)	
	city *	VARCHAR(255)	
	bio *	TEXT	
	expertise *	LONGTEXT	
	skills *	LONGTEXT	
	experience_years	INT UNSIGNED	
	capacity	INT UNSIGNED	
	accepting_requests	TINYINT(1)	
	review_modes *	LONGTEXT	
	portfolio_url *	VARCHAR(2048)	
	evaluator_status	VARCHAR(255)	
	moderation_note *	TEXT	
	profile_photo_path *	VARCHAR(255)	
UQ	google_id *	VARCHAR(255)	
UQ	username *	VARCHAR(80)	

project_user

Additional group members; owner is projects.user_id.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

projects

Thesis details, student owner, visibility and review settings.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
	summary	TEXT	
	school *	VARCHAR(255)	
	course *	VARCHAR(255)	
	category	VARCHAR(255)	
	technologies *	LONGTEXT	
	feedback_needed	TEXT	
	status	VARCHAR(255)	
	moderation_note *	TEXT	
	deadline *	DATE	
	is_public	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	pdf_access	VARCHAR(24)	
	review_mode	VARCHAR(24)	
	reviewer_limit	TINYINT UNSIGNED	

document_versions

One record per thesis PDF version; files stay in private storage.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	original_name	VARCHAR(255)	
	path	VARCHAR(1024)	
	mime_type	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	version	INT UNSIGNED	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

users.role separates student, evaluator and admin accounts. There is no separate students or evaluators account table. projects.user_id is the owner; project_user stores additional group members.

document_versions.id identifies the PDF record. document_versions.version is the visible version number (1, 2, …). The path points to a private file; the database stores metadata, not the PDF contents.

Reviews, feedback and evaluator applications

Full columns • follow blue FK arrows to the parent table named on the same row.

PK Primary key: unique row ID **FK → Enforced foreign key: child column → parent key**

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

review_requests

One evaluator assignment for one project and one exact PDF.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	evaluator_id	BIGINT UNSIGNED	→ users.id
FK	document_version_id	BIGINT UNSIGNED	→ document_versions.id
FK	requested_by	BIGINT UNSIGNED	→ users.id
	status	VARCHAR(255)	
	message	TEXT	
	preferred_date *	DATE	
	agreed_date *	DATE	
	accepted_at *	TIMESTAMP	
	completed_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	final_summary *	TEXT	
	output_path *	VARCHAR(1024)	
	output_original_name *	VARCHAR(255)	
	output_size *	BIGINT UNSIGNED	

feedback

Page / section comments attached to a review request.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	review_request_id	BIGINT UNSIGNED	→ review_requests.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	section *	VARCHAR(255)	
	page *	INT UNSIGNED	
	type	VARCHAR(255)	
	body	TEXT	
	priority	VARCHAR(255)	
	addressed	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

ratings

A student rates an evaluator for a completed review.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	review_request_id	BIGINT UNSIGNED	→ review_requests.id
FK	user_id	BIGINT UNSIGNED	→ users.id
FK	evaluator_id	BIGINT UNSIGNED	→ users.id
	rating	TINYINT UNSIGNED	
	comment *	TEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

evaluator_verifications

Submitted application snapshot and administrator decision.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	profile_snapshot	LONGTEXT	
	profile_fingerprint	CHAR(64)	
	background_type	VARCHAR(255)	
	organization *	VARCHAR(255)	
	background_summary	TEXT	
	evidence_type	VARCHAR(255)	
	evidence_url *	VARCHAR(500)	
	evidence_explanation *	TEXT	
	reference_name *	VARCHAR(255)	
	reference_email *	VARCHAR(255)	
	reference_relationship *	VARCHAR(300)	
	work_sample_url	VARCHAR(500)	
	work_sample_explanation	TEXT	
	sample_review	TEXT	
	attestation	TINYINT(1)	
	status	VARCHAR(255)	
FK	reviewer_id *	BIGINT UNSIGNED	→ users.id
	reviewed_at *	TIMESTAMP	
	review_notes *	TEXT	
	checks *	LONGTEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

evaluator_evidence

Private files supporting an evaluator application.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	evaluator_verification_id	BIGINT UNSIGNED	→ evaluator_verifications.id
	path	VARCHAR(1024)	
	original_name	VARCHAR(255)	
	mime_type	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

invitations

Evaluator invitation; records the administrator who sent it.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	email	VARCHAR(255)	
UQ	token_hash	VARCHAR(64)	
FK	invited_by	BIGINT UNSIGNED	→ users.id
	expires_at	TIMESTAMP	
	accepted_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

review_requests links project_id → projects.id, evaluator_id → users.id, document_version_id → document_versions.id, and requested_by → users.id.
Names and titles are retrieved with JOIN.

evaluator_verifications.user_id is the applicant; reviewer_id is the administrator who reviews the application. evaluator_evidence points to a submitted application, not directly to a user.

Messaging, community chat and learning

Full columns • private conversations and saved tutorials connect back to user accounts.

PK Primary key: unique row ID **FK → Enforced foreign key: child column → parent key**

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

direct_conversations

A private conversation between two user accounts.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	lower_user_id	BIGINT UNSIGNED	→ users.id
FK	higher_user_id	BIGINT UNSIGNED	→ users.id
	last_message_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

direct_messages

Messages and review events inside a conversation.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	direct_conversation_id	BIGINT UNSIGNED	→ direct_conversations.id
FK	sender_id	BIGINT UNSIGNED	→ users.id
FK	recipient_id	BIGINT UNSIGNED	→ users.id
	body	TEXT	
	read_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	kind	VARCHAR(30)	
FK	review_request_id *	BIGINT UNSIGNED	→ review_requests.id
	review_event *	VARCHAR(30)	
UQ	review_event_key *	VARCHAR(150)	

direct_message_attachments

Image / document metadata for a private message.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	direct_message_id	BIGINT UNSIGNED	→ direct_messages.id
	name	VARCHAR(180)	
UQ	path	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	mime_type	VARCHAR(127)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

community_chat_messages

Community messages from accounts or guest visitors.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	author_key	VARCHAR(80)	
	author_name	VARCHAR(255)	
	is_guest	TINYINT(1)	
	body	TEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

community_chat_presences

Who was recently present in community chat.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
UQ	viewer_key	VARCHAR(80)	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	guest_name *	VARCHAR(255)	
	last_seen_at	TIMESTAMP	

tutorials

Learning guides / videos and their optional author.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
UQ	slug	VARCHAR(255)	
	category	VARCHAR(255)	
	excerpt	TEXT	
	body	LONGTEXT	
	reading_minutes	INT UNSIGNED	
	is_published	TINYINT(1)	
	featured	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	youtube_video_id *	VARCHAR(11)	
	video_creator *	VARCHAR(120)	

tutorial_user

Saved tutorials; links accounts to tutorial records.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	tutorial_id	BIGINT UNSIGNED	→ tutorials.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

direct_conversations stores the two participants. direct_messages stores sender, recipient and message content. Attachments belong to a message.
An optional review_request_id links review events to the relevant review.

tutorial_user is a many-to-many link: an account can save many tutorials, and a tutorial can be saved by many accounts. Its foreign keys are tutorial_id and user_id.

Notifications, reports and administration

Full columns • blue arrows are enforced keys; amber references are resolved by application code.

PK Primary key: unique row ID FK → Enforced foreign key: child column → parent key

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

activities

Notifications: recipient, actor and destination URL.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
	body *	TEXT	
	url	VARCHAR(2048)	
	read_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
UQ	review_event_key *	VARCHAR(150)	
FK	actor_id *	BIGINT UNSIGNED	→ users.id

admin_actions

Immutable administration decisions and status changes.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	actor_id *	BIGINT UNSIGNED	→ users.id
	action	VARCHAR(64)	
	target_type	VARCHAR(32)	
REF	target_id	BIGINT UNSIGNED	⇢ record selected by target_type
	before_values *	LONGTEXT	
	after_values *	LONGTEXT	
	metadata *	LONGTEXT	
	reason *	TEXT	
	created_at	TIMESTAMP	

reports

Moderation reports about a project or user.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	reportable_type	VARCHAR(255)	
REF	reportable_id	BIGINT UNSIGNED	⇢ projects.id / users.id (type selects)
	reason	VARCHAR(255)	
	details *	TEXT	
	status	VARCHAR(255)	
	admin_note *	TEXT	
	action *	VARCHAR(255)	
	resolved_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

sensitive_file_accesses

Immutable metadata of authorized file download requests.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	actor_id *	BIGINT UNSIGNED	→ users.id
	actor_role	VARCHAR(16)	
	file_type	VARCHAR(32)	
REF	file_id	BIGINT UNSIGNED	⇢ record selected by file_type
REF	project_id *	BIGINT UNSIGNED	⇢ projects.id (audit reference)
REF	verification_id *	BIGINT UNSIGNED	⇢ evaluator_verifications.id (audit)
	version *	INT UNSIGNED	
	created_at	TIMESTAMP	

activities.user_id is the notification recipient; actor_id is the person who triggered it. reports.user_id is the reporter. reportable_type and reportable_id identify the reported project or user without a database foreign key.

admin_actions.actor_id is the administrator. target_type + target_id identify the affected record. In sensitive_file_accesses, file_type selects the meaning of file_id; project_id and verification_id are audit references. These IDs intentionally survive some record deletions.

Laravel support tables

Full columns • these support sessions, caching, background jobs and schema history.

PK Primary key: unique row ID **FK → Enforced foreign key: child column → parent key**

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

sessions

Browser login sessions and optional account reference.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	VARCHAR(255)	
REF	user_id *	BIGINT UNSIGNED	⇢ users.id (session reference)
	ip_address *	VARCHAR(45)	
	user_agent *	TEXT	
	payload	LONGTEXT	
	last_activity	INT	

password_reset_tokens

Password-reset tokens indexed by email address.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	email	VARCHAR(255)	⇢ users.email (account lookup)
	token	VARCHAR(255)	
	created_at *	TIMESTAMP	

cache

Temporary cached values and expiry time.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	key	VARCHAR(255)	
	value	MEDIUMTEXT	
	expiration	INT	

cache_locks

Temporary application locks.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	key	VARCHAR(255)	
	owner	VARCHAR(255)	
	expiration	INT	

jobs

Queued background work.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	queue	VARCHAR(255)	
	payload	LONGTEXT	
	attempts	TINYINT UNSIGNED	
	reserved_at *	INT UNSIGNED	
	available_at	INT UNSIGNED	
	created_at	INT UNSIGNED	

job_batches

Progress and counters for grouped jobs.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	VARCHAR(255)	
	name	VARCHAR(255)	
	total_jobs	INT	
	pending_jobs	INT	
	failed_jobs	INT	
	failed_job_ids	LONGTEXT	
	options *	MEDIUMTEXT	
	cancelled_at *	INT	
	created_at	INT	
	finished_at *	INT	

failed_jobs

Background jobs that failed.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
UQ	uuid	VARCHAR(255)	
	connection	TEXT	
	queue	TEXT	
	payload	LONGTEXT	
	exception	LONGTEXT	
	failed_at	TIMESTAMP	

migrations

Schema changes already applied; not user activity.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	INT UNSIGNED	
	migration	VARCHAR(255)	
	batch	INT	

sessions.user_id is an application reference to users.id, not an enforced foreign key in this schema. password_reset_tokens.email is an account lookup key; the same email is used to find the account.

cache, cache_locks, jobs, job_batches and failed_jobs have no enforced foreign keys. migrations records schema changes already applied. It is unrelated to the administrator action history.

All Analista tables and their connections

Each table row below is a COLUMN definition. Arrows name the table and column it connects to; no live user records are included.

PK Primary key: unique row ID **FK → Enforced foreign key: child column → parent key**

REF ⇢ Application / audit reference, not a database foreign key

* = NULL allowed • UQ = unique column / index

29 tables • 275 columns • 35 enforced foreign keys • application-only references marked REF ⇢

users

All accounts: students, evaluators and administrators.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	name	VARCHAR(255)	
UQ	email	VARCHAR(255)	
	email_verified_at *	TIMESTAMP	
	password *	VARCHAR(255)	
	remember_token *	VARCHAR(100)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	role	VARCHAR(255)	
	is_suspended	TINYINT(1)	
	headline *	VARCHAR(255)	
	city *	VARCHAR(255)	
	bio *	TEXT	
	expertise *	LONGTEXT	
	skills *	LONGTEXT	
	experience_years	INT UNSIGNED	
	capacity	INT UNSIGNED	
	accepting_requests	TINYINT(1)	
	review_modes *	LONGTEXT	
	portfolio_url *	VARCHAR(2048)	
	evaluator_status	VARCHAR(255)	
	moderation_note *	TEXT	
	profile_photo_path *	VARCHAR(255)	
UQ	google_id *	VARCHAR(255)	
UQ	username *	VARCHAR(80)	

projects

Thesis details, student owner, visibility and review settings.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
	summary	TEXT	
	school *	VARCHAR(255)	
	course *	VARCHAR(255)	
	category	VARCHAR(255)	
	technologies *	LONGTEXT	
	feedback_needed	TEXT	
	status	VARCHAR(255)	
	moderation_note *	TEXT	
	deadline *	DATE	
	is_public	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	pdf_access	VARCHAR(24)	
	review_mode	VARCHAR(24)	
	reviewer_limit	TINYINT UNSIGNED	

document_versions

One record per thesis PDF version; files stay in private storage.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	original_name	VARCHAR(255)	
	path	VARCHAR(1024)	
	mime_type	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	version	INT UNSIGNED	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

project_user

Additional group members; owner is projects.user_id.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

review_requests

One evaluator assignment for one project and one exact PDF.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	project_id	BIGINT UNSIGNED	→ projects.id
FK	evaluator_id	BIGINT UNSIGNED	→ users.id
FK	document_version_id	BIGINT UNSIGNED	→ document_versions.id
FK	requested_by	BIGINT UNSIGNED	→ users.id
	status	VARCHAR(255)	
	message	TEXT	
	preferred_date *	DATE	
	agreed_date *	DATE	
	accepted_at *	TIMESTAMP	
	completed_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	final_summary *	TEXT	
	output_path *	VARCHAR(1024)	
	output_original_name *	VARCHAR(255)	
	output_size *	BIGINT UNSIGNED	

feedback

Page / section comments attached to a review request.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	review_request_id	BIGINT UNSIGNED	→ review_requests.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	section *	VARCHAR(255)	
	page *	INT UNSIGNED	
	type	VARCHAR(255)	
	body	TEXT	
	priority	VARCHAR(255)	
	addressed	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

ratings

A student rates an evaluator for a completed review.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	review_request_id	BIGINT UNSIGNED	→ review_requests.id
FK	user_id	BIGINT UNSIGNED	→ users.id
FK	evaluator_id	BIGINT UNSIGNED	→ users.id
	rating	TINYINT UNSIGNED	
	comment *	TEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

evaluator_verifications

Submitted application snapshot and administrator decision.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	profile_snapshot	LONGTEXT	
	profile_fingerprint	CHAR(64)	
	background_type	VARCHAR(255)	
	organization *	VARCHAR(255)	
	background_summary	TEXT	
	evidence_type	VARCHAR(255)	
	evidence_url *	VARCHAR(500)	
	evidence_explanation *	TEXT	
	reference_name *	VARCHAR(255)	
	reference_email *	VARCHAR(255)	
	reference_relationship *	VARCHAR(300)	
	work_sample_url	VARCHAR(500)	
	work_sample_explanation	TEXT	
	sample_review	TEXT	
	attestation	TINYINT(1)	
	status	VARCHAR(255)	
FK	reviewer_id *	BIGINT UNSIGNED	→ users.id
	reviewed_at *	TIMESTAMP	
	review_notes *	TEXT	
	checks *	LONGTEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

evaluator_evidence

Private files supporting an evaluator application.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	evaluator_verification_id	BIGINT UNSIGNED	→ evaluator_verifications.id
	path	VARCHAR(1024)	
	original_name	VARCHAR(255)	
	mime_type	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

direct_conversations

A private conversation between two user accounts.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	lower_user_id	BIGINT UNSIGNED	→ users.id
FK	higher_user_id	BIGINT UNSIGNED	→ users.id
	last_message_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

direct_messages

Messages and review events inside a conversation.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	direct_conversation_id	BIGINT UNSIGNED	→ direct_conversations.id
FK	sender_id	BIGINT UNSIGNED	→ users.id
FK	recipient_id	BIGINT UNSIGNED	→ users.id
	body	TEXT	
	read_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	kind	VARCHAR(30)	
FK	review_request_id *	BIGINT UNSIGNED	→ review_requests.id
	review_event *	VARCHAR(30)	
UQ	review_event_key *	VARCHAR(150)	

direct_message_attachments

Image / document metadata for a private message.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	direct_message_id	BIGINT UNSIGNED	→ direct_messages.id
	name	VARCHAR(180)	
UQ	path	VARCHAR(255)	
	size	BIGINT UNSIGNED	
	mime_type	VARCHAR(127)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

community_chat_messages

Community messages from accounts or guest visitors.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	author_key	VARCHAR(80)	
	author_name	VARCHAR(255)	
	is_guest	TINYINT(1)	
	body	TEXT	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

community_chat_presences

Who was recently present in community chat.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
UQ	viewer_key	VARCHAR(80)	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	guest_name *	VARCHAR(255)	
	last_seen_at	TIMESTAMP	

tutorials

Learning guides / videos and their optional author.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id *	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
UQ	slug	VARCHAR(255)	
	category	VARCHAR(255)	
	excerpt	TEXT	
	body	LONGTEXT	
	reading_minutes	INT UNSIGNED	
	is_published	TINYINT(1)	
	featured	TINYINT(1)	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
	youtube_video_id *	VARCHAR(11)	
	video_creator *	VARCHAR(120)	

tutorial_user

Saved tutorials; links accounts to tutorial records.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	tutorial_id	BIGINT UNSIGNED	→ tutorials.id
FK	user_id	BIGINT UNSIGNED	→ users.id
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

invitations

Evaluator invitation; records the administrator who sent it.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	email	VARCHAR(255)	
UQ	token_hash	VARCHAR(64)	
FK	invited_by	BIGINT UNSIGNED	→ users.id
	expires_at	TIMESTAMP	
	accepted_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

NOTIFICATIONS AND ADMINISTRATION

activities

Notifications: recipient, actor and destination URL.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	title	VARCHAR(255)	
	body *	TEXT	
	url	VARCHAR(2048)	
	read_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	
UQ	review_event_key *	VARCHAR(150)	
FK	actor_id *	BIGINT UNSIGNED	→ users.id

admin_actions

Immutable administration decisions and status changes.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	actor_id *	BIGINT UNSIGNED	→ users.id
	action	VARCHAR(64)	
	target_type	VARCHAR(32)	
REF	target_id	BIGINT UNSIGNED	⇢ record selected by target_type
	before_values *	LONGTEXT	
	after_values *	LONGTEXT	
	metadata *	LONGTEXT	
	reason *	TEXT	
	created_at	TIMESTAMP	

sessions

Browser login sessions and optional account reference.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	VARCHAR(255)	
REF	user_id *	BIGINT UNSIGNED	⇢ users.id (session reference)
	ip_address *	VARCHAR(45)	
	user_agent *	TEXT	
	payload	LONGTEXT	
	last_activity	INT	

password_reset_tokens

Password-reset tokens indexed by email address.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	email	VARCHAR(255)	⇢ users.email (account lookup)
	token	VARCHAR(255)	
	created_at *	TIMESTAMP	

cache

Temporary cached values and expiry time.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	key	VARCHAR(255)	
	value	MEDIUMTEXT	
	expiration	INT	

cache_locks

Temporary application locks.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	key	VARCHAR(255)	
	owner	VARCHAR(255)	
	expiration	INT	

reports

Moderation reports about a project or user.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	user_id	BIGINT UNSIGNED	→ users.id
	reportable_type	VARCHAR(255)	
REF	reportable_id	BIGINT UNSIGNED	⇢ projects.id / users.id (type selects)
	reason	VARCHAR(255)	
	details *	TEXT	
	status	VARCHAR(255)	
	admin_note *	TEXT	
	action *	VARCHAR(255)	
	resolved_at *	TIMESTAMP	
	created_at *	TIMESTAMP	
	updated_at *	TIMESTAMP	

sensitive_file_accesses

Immutable metadata of authorized file download requests.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
FK	actor_id *	BIGINT UNSIGNED	→ users.id
	actor_role	VARCHAR(16)	
	file_type	VARCHAR(32)	
REF	file_id	BIGINT UNSIGNED	⇢ record selected by file_type
REF	project_id *	BIGINT UNSIGNED	⇢ projects.id (audit reference)
REF	verification_id *	BIGINT UNSIGNED	⇢ evaluator_verifications.id (audit)
	version *	INT UNSIGNED	
	created_at	TIMESTAMP	

jobs

Queued background work.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
	queue	VARCHAR(255)	
	payload	LONGTEXT	
	attempts	TINYINT UNSIGNED	
	reserved_at *	INT UNSIGNED	
	available_at	INT UNSIGNED	
	created_at	INT UNSIGNED	

job_batches

Progress and counters for grouped jobs.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	VARCHAR(255)	
	name	VARCHAR(255)	
	total_jobs	INT	
	pending_jobs	INT	
	failed_jobs	INT	
	failed_job_ids	LONGTEXT	
	options *	MEDIUMTEXT	
	cancelled_at *	INT	
	created_at	INT	
	finished_at *	INT	

failed_jobs

Background jobs that failed.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	BIGINT UNSIGNED	
UQ	uuid	VARCHAR(255)	
	connection	TEXT	
	queue	TEXT	
	payload	LONGTEXT	
	exception	LONGTEXT	
	failed_at	TIMESTAMP	

migrations

Schema changes already applied; not user activity.

KEY	COLUMN	SQL TYPE	CONNECTS TO
PK	id	INT UNSIGNED	
	migration	VARCHAR(255)	
	batch	INT	